Taller personalizado de Programación y Robótica en CMM BML 2023 – CL6 y CL7



[R] Servos 0º - 180º Intro y Calibración

POO : programación orientada a Objetos

Clase Servo : construcción de la clase y uso

29 Marzo 2023 ok

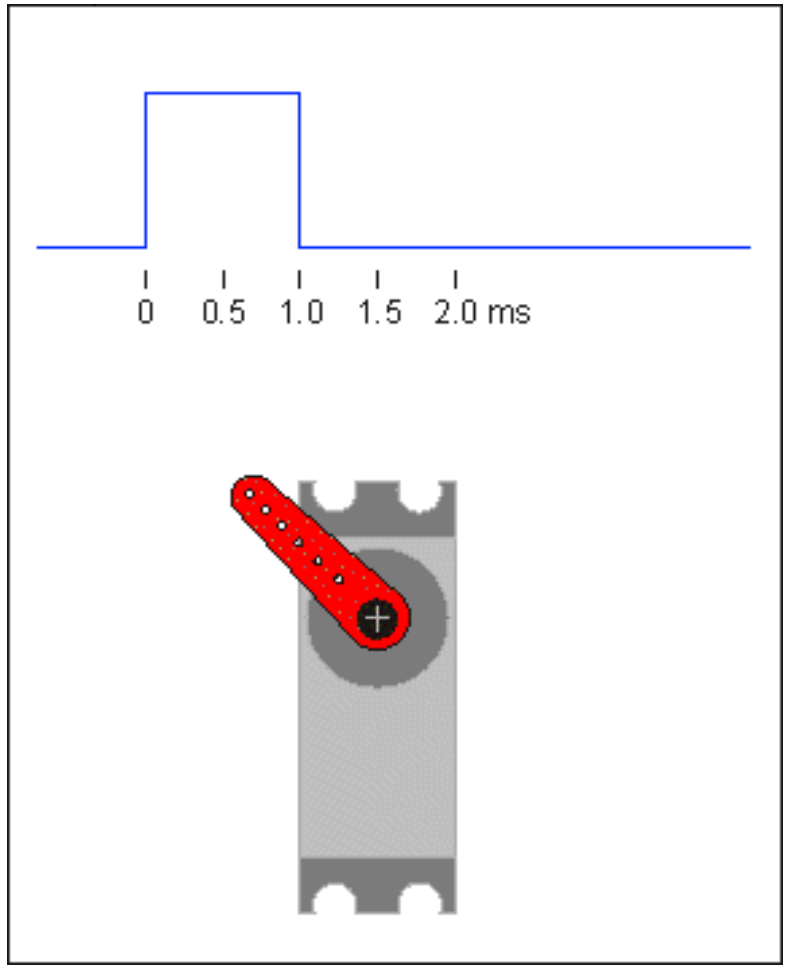
Voluntario : J.C. Santamaria

Este obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional.

Clase 6-[R] – **Servos 0º - 180º Introducción**

(hay otros servos de movimiento continuo que veremos en otra clase)

- **Un Servo es un mecanismo que posiciona un brazo en una posición fija entre 0º y 180º .** Son baratos y por eso se usan mucho en robotica
- Internamente tiene un motor dc, un sensor de posición (potenciómetro) y un circuito de realimentación. La comunicación de la posición deseada se realiza mediante la transmisión de un señal pulsada (PWM) con periodo de 20ms (= 50Hz). **El ancho del pulso PWM determina la posición del servo.**
- En general, en todos los modelos:
  - Un pulso entre **500-1000 us corresponde con 0º**
  - Un pulso de **1500 ms corresponde con 90º** (punto neutro)
  - Un pulso entre **2000-2500us corresponde con 180º**
- **Los servos** (especialmente si son baratos) **hay que calibrarlos siempre**



Seguir enlace para saber mas : SG90 especificaciones

CC BY NC SA

Clase 6-[R] – Demo Calibrado de un servo SG90

Conexión del Servo a Pico W (funcionaria Ok en un PICO noW)

- Rojo -> VBUS (5volt) pin 40
- Negro -> GND (hay varios pines GND)
- Naranja -> GPIO15 pin 20 (se puede usar otro) – señal PWM

Idea: Introducir los microsegundos de estar en "1" y ver la posición del servo en grados. Habrá posiciones limite

1. Inicialización del pin del servo a PWM 50Hz (20ms)
2. Ponemos los microsegundo iniciales a 1500: suele ser la posición de 90º
3. **Bucle infinito**
   1. con input de posiciones en microsegundos entre los valores mas probables
   2. Cambiar el "duty" del PWM al nuevo valor de microsegundos
   3. y ver posición del brazo del servo

BMMR_CL5_servo_calibra_v1_0.py

Valido PicoW & Pico

```
from machine import Pin, PWM

EXTERNAL_SERVO_PIN = 15
pwm_servo = PWM(Pin(EXTERNAL_SERVO_PIN))
pwm_servo.freq(50)

us_servo = 1500  # usually correspond to 90º

while (True):
    pwm_servo.duty_ns(us_servo*1000)
    us = input("microseg servo (700 a 2600)= ")
    us_servo = int(us)
```


CC BY NC SA

Clase 6-[R] – Demo Posicionar los grados en un servo SG90 – programa BMMR_CL6_servo_in_deg_v1_2.py

30'

Conexión del Servo -> Pico

- Rojo -> VSYS (5volt) pin 39 // Negro -> GND (hay varios pines GND) // Naranja -> GPIO15 pin 20 (se puede usar otro) –> señal PWM

Idea: Introducir los grados donde queremos que se posicione el servo. Con los datos de calibración haremos una función que pase de grados a nanosegundos

1. Inicialización del pin del servo a PWM 50Hz (20ms)
2. Función **ConverDeg_us(deg)**

   Usaremos los parámetros que hemos visto en calibración para 0ª, 90º y 180º, para convertir grados en nanosegundos

   Es una función de "mapeo" : son muy usuales en robotica, el algoritmo es siempre el mismo (ver codigo)
3. Ponemos los grados del servo iniciales a 90º + convertir a nanosegundos
4. **Bucle infinito**
   1. Posicionar servo, con info de nanosegundos
   2. input de Grados de 0-180ª
   3. Usar función ConvertDeg_us, para pasar de grados deseados a nanosegundos
   4. (debug) Mostrar grados y microsegundos (= nanosegundos x 1000)

CC BY NC SA

Clase 6-[R] – Demo Posicionar los grados en un servo SG90 – programa BMMR_CL6_servo_in_deg_v1_2.py

```
from machine import Pin, PWM


SERVO_CALIBRA_0DEG = 700 # microsec correspond to 90
SERVO_CALIBRA_90DEG = 1500 # microsec correspond to 90
SERVO_CALIBRA_180DEG = 2500  # microsec correspond to 180
EXTERNAL_SERVO_PIN = 15
SERVO_PWM_FREQ = 50


def ConverDeg_us(deg):
    if deg < 0 :
        deg = 0
    if deg > 180:
        deg = 180
    if deg < 90:
        return int(SERVO_CALIBRA_0DEG + deg * (SERVO_CALIBRA_90DEG - SERVO_CALIBRA_0DEG) / 90)
    else:
        return int(SERVO_CALIBRA_90DEG + (deg-90) * (SERVO_CALIBRA_180DEG - SERVO_CALIBRA_90DEG) / 90)
```

```
pwm_servo = PWM(Pin(EXTERNAL_SERVO_PIN)) # servo object creation
pwm_servo.freq(SERVO_PWM_FREQ)


deg_servo = 90  # initial position
us_servo = ConverDeg_us(deg_servo)


while (True):
    pwm_servo.duty_ns(us_servo*1000)
    deg_servo = int(input("Deg servo (0 a 180)= "))
    us_servo = ConverDeg_us(deg_servo)
    # Poner un comentario para no mostar la siguiente informacion de debug
    if deg_servo >= 0 and deg_servo <= 180:
        print('Grados = ', deg_servo, 'corresponde a microsegundos = ', us_servo)
    else:
        print ('Error in Deg, out of range. Deg limitted to 0º or 180º')
```


CC BY NC SA



Clase 7-[RyP] – Programación Orientada a Objetos: Algunos conceptos

Las entidades de la vida real se pueden modelar en Python como objetos, y a eso se le denomina **programación orientada a objetos POO.**

La definición de un modelo para ciertos objetos se denomina clase. —> es solo la definición **NO** crea ningún objeto, es como un molde

Las clases tienen atributos y métodos

Los atributos o propiedades de las clases, especifican con la ayuda de las variables asociadas a una clase determinada. Estas variables se denominan propiedades o atributos.

Las operaciones computacionales asociadas con las entidades se expresan como las definiciones de funciones asociadas con la clase dada. Estas funciones se denominan métodos.

La definición de una clase incluye —> VER ejemplo

- **Atributos**. Los atributos se "crean" como variables dentro del cuerpo de una clase.
- **Constructor**: El método constructor se define mediante la función **__init__()** establece explícitamente los valores predeterminados de los atributos o asigna los valores de los atributos esenciales a través de argumentos
- y la definición de los **métodos específicos de la clase, que harán operaciones subiendo un nivel de abstracción**, por ejemplo en clase Servo "GiraGrados" que hace girar el eje del servo un número de grados
- **Sintaxis:** dentro **self.**atributo, fuera **nombre_clase**.atributo, idem con métodos —> VER ejemplo en pagina siguiente

(es un ejemplo en micropython, pero en esto Python y micropython funcionan igual)

CC BY NC SA

Clase 7-[R] – Demo clase para controlar un Servo 0-180

```
class Servo:
    def __init__(self, c0degus=700, c90degus=1500, c180degus=2500, pin=15, freq=50):
        self.pwm = PWM(Pin(pin))
        self.pwm.freq(freq)
        self.MIN_DUTY_ns = c0degus * 1000 # time pulse for 0 deg
        self.CEN_DUTY_ns = c90degus * 1000 # time pulse for 90 deg
        self.MAX_DUTY_ns = c180degus * 1000 # time pulse for 180 deg


    def RotateDeg(self, deg, show=False):
        if deg < 0:
            deg = 0
        if deg > 180:
            deg = 180
        if deg < 90:
            duty_ns = int(self.MIN_DUTY_ns + deg * (self.CEN_DUTY_ns - self.MIN_DUTY_ns) / 90)
        else:
            duty_ns = int(self.CEN_DUTY_ns + (deg-90) * (self.MAX_DUTY_ns - self.CEN_DUTY_ns) / 90)
```

BMMR_CL7_servo_deg_class_v1_1_1.py

```
class Servo:
(…)
if show:
            print(f"Grados: {deg} - Duty nanosconds : {duty_ns}")
        self.pwm.duty_ns(duty_ns)


    def ShowDuty_ns(self):
        return self.pwm.duty_ns()
```


CC BY NC SA

[Robótica]#5–Pines de la Pico W

Necesario tener en papel

Es muy recomendable tener en papel el dibujo con los nombres de los pines de la tarjeta, porque es como los nombraras en los programas

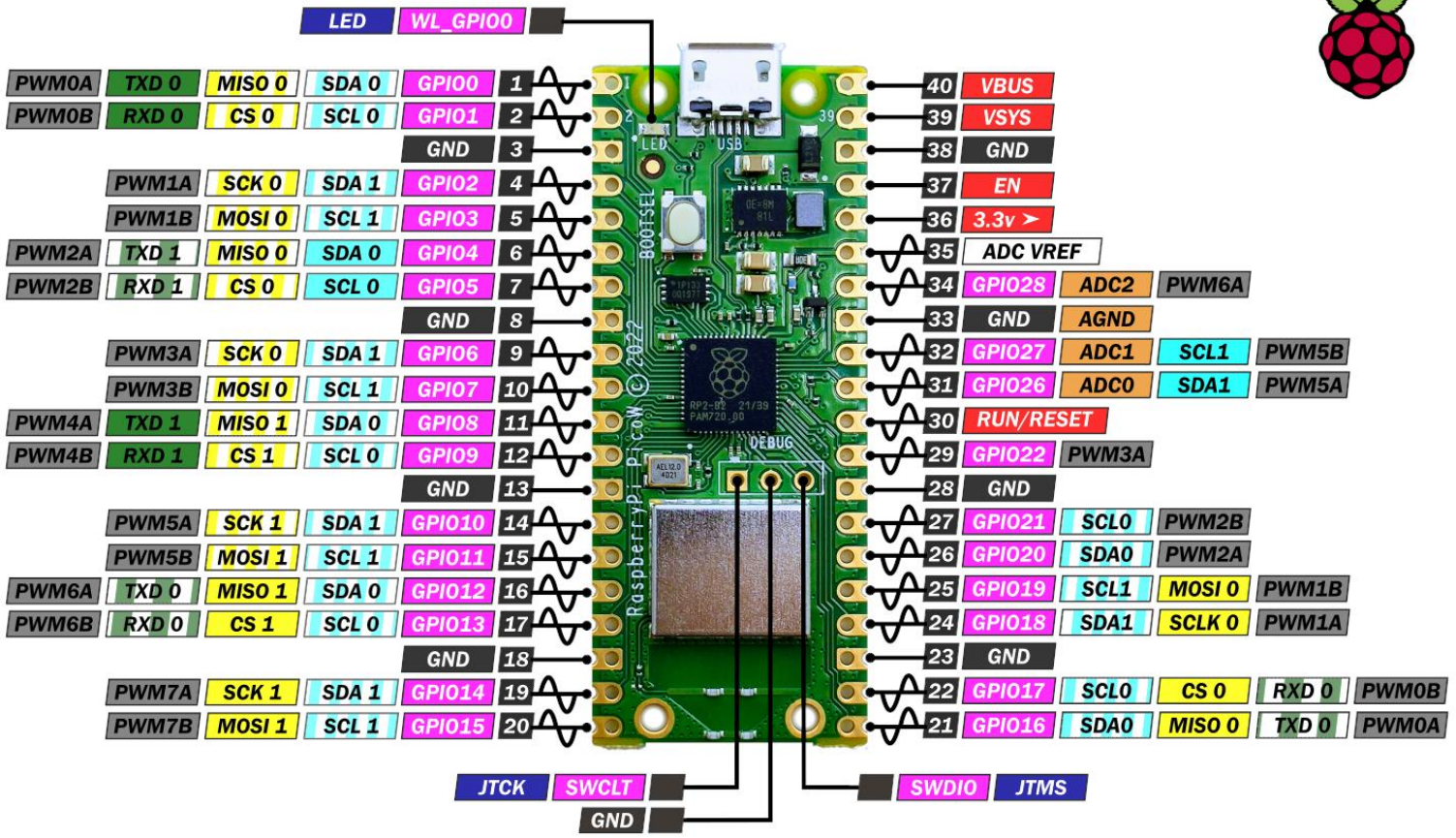
Lo usaras muchas veces.

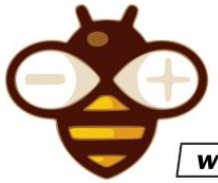
Lo básico

GPIOnn : pin de entrada o salida de propósito general

GND : terminal negativo

3.3V terminal positivo

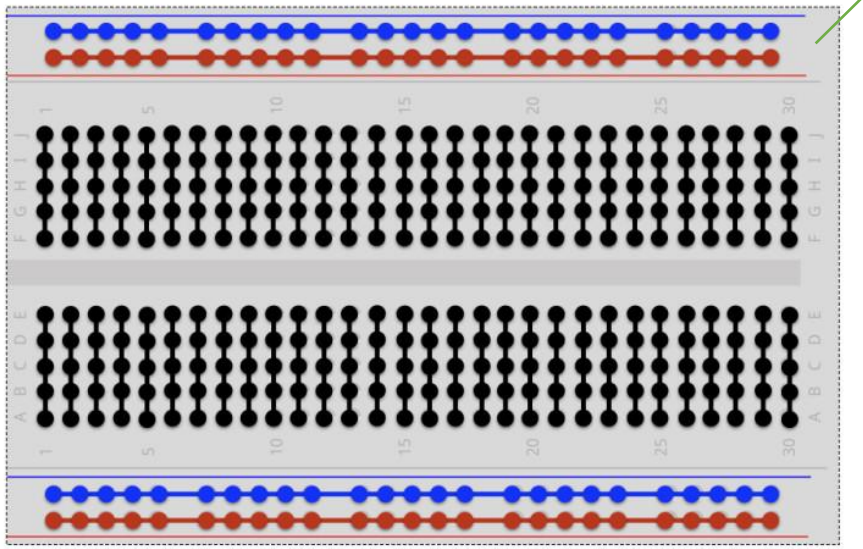




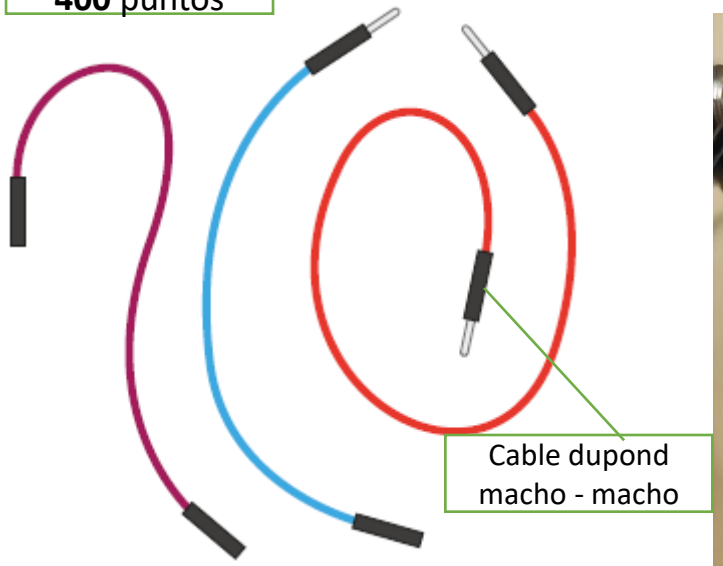
www.mischianti.org

Internal board function pins	
IP Used in ADC mode (ADC3) to measure VSYS/3	GPIO29
OP Connected to user LED	GPIO25
IP VBUS sense - high if VBUS is present, else low	GPIO24
OP Controls the on-board SMPS Power Save pin	GPIO23

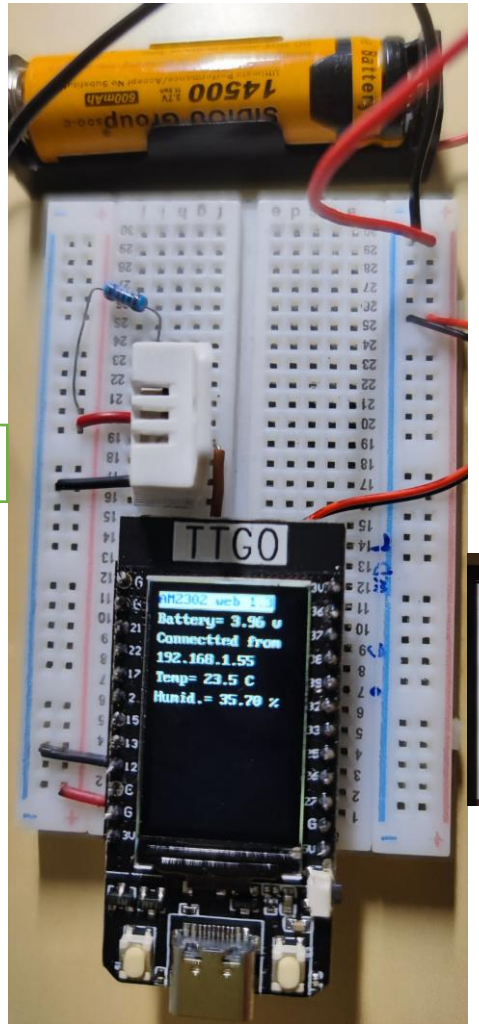
[Robótica]#3–Protoboard ¿qué es? Uso en prototipos (en ingles breadboard)



Protoboard de **400** puntos

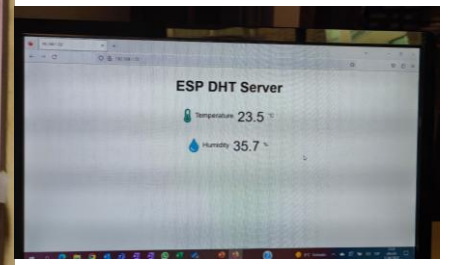


Cable dupond macho - macho



Ejemplo real:
Servidor web de temperatura y humedad,

hecho con ESP32 con display y sensor AM2302



Una **Protoboard** permite conexiones entre componentes electrónicos con agujeros a distancia standard 0.1”=2.5 mm, conectados (ver esquema)

Las conexiones que no hace la **Protoboard**, se hacen con cables “dupond” macho – macho o con cable rígido en forma de “grapa”

Hay 2 x 2 **líneas de alimentación**, que permiten alimentar componentes dos voltajes, como 5 y 3.3 voltios

Hay 3 **tamaños** de Protoboard mas comunes : 170, 400 y 832

CC BY NC SA